ジオメトリック・インテリジェンス

ケーススタディ補助文書 03

実装スクリプト集

― Eight Eyesの多様体：8カ国インテリジェンス統合のためのジオメトリック・インテリジェンス理論の国家安全保障への適用 ―

本書は、ケーススタディ・ノベル「Eight Eyesの多様体」の数学について、
実行して出力を確かめた最小限の参照実装と、実装上の注意を示す。

Étale Cohomology

2026 年 10 月

License: CC BY 4.0

目次

1　はじめに 2

2　整合性の半径（第6章・第19章） 2

3　Lie括弧（第9章） 2

4　臨界減速の兆候（第8章） 3

5　相互情報量、データ処理不等式、KL情報量（第12章） 3

6　偏りのある確率的勾配降下法（第13章） 4

付録 A　参照実装 5

付録 B　著者による関連記事 6

第1章　はじめに

本編のコードは、物語の中で計算の流れを示すための抜粋で、関数名の多くは説明のための架空の名前である。本書では、本作で新しく登場した考え方について、理論値と照らし合わせられる最小限の参照実装を示す。すべてPython（NumPy）で実行し、出力を確認している。

第2章　整合性の半径（第6章・第19章）

3体のエージェントが同じ量を推定し、辺で一致すべきと結ばれた、最も簡単な層（制限写像が恒等写像）で、整合性の半径と辺ごとの食い違いを計算する。

```
import numpy as np
# 3体のエージェント（ノード）が同じ量（例：ある地域の緊張度）を推定し、辺で「一致すべき」と結ばれている
# 制限写像は恒等写像とする最も簡単な層。整合性の半径 =
↪  全体として整合な割り当てからの最大のずれ（ここでは辺ごとの食い違いの最大）
def radius(x, edges):
    gaps={e:abs(x[e[0]]-x[e[1]]) for e in edges}
    return max(gaps.values()), gaps
edges=[(0,1),(1,2),(0,2)]
for name,x in [("整合している",[0.40,0.42,0.41]),("1体が食い違う",[0.40,0.42,0.95])]:
    r,g=radius(np.array(x),edges)
    print(f"{name}: 整合性の半径={r:.2f}  辺ごとの食い違い={ {k:round(float(v),2) for k,v in g.items()} }")
```

出力：整合している場合は半径0.02、1体が食い違う場合は半径0.55となり、食い違いはそのエージェントにつながる辺（(1,2)と(0,2)）に集中する。
実装上の注意：実際には、制限写像を行列とし、層ラプラシアンや整合性のフィルトレーションで計算する。Neural Sheaf Diffusionでは制限写像を学習する。逸脱は「調べるべき場所」の合図として扱い、事実の確定は法にもとづく手続きに委ねる。

第3章　Lie括弧（第9章）

```
import numpy as np
# ベクトル場 X, Y のリー括弧 [X, Y](z) = J_Y(z) X(z) - J_X(z) Y(z) を、中心差分のヤコビ行列で計算する
def jacobian(F, z, eps=1e-6):
    n = len(z); J = np.zeros((n, n))
    for k in range(n):
        e = np.zeros(n); e[k] = eps
        J[:, k] = (F(z + e) - F(z - e)) / (2*eps)
    return J
def lie_bracket(X, Y, z):
    return jacobian(Y, z) @ X(z) - jacobian(X, z) @ Y(z)
# 例：平面の回転の場 X と、x 方向の一様な場 Y
X = lambda z: np.array([-z[1], z[0]])
Y = lambda z: np.array([1.0, 0.0])
z = np.array([0.3, -0.2])
print("[X, Y](z) =", np.round(lie_bracket(X, Y, z), 6))   # 理論値は (0, -1)
# 可換な場どうしでは 0 になることの確認
Z = lambda z: np.array([2.0, 0.5])
```

```
print("[Y, Z](z) =", np.round(lie_bracket(Y, Z, z), 6))
```

出力：[X, Y](z) = [0. -1.]、[Y, Z](z) = [0. 0.]（理論値と一致）。
実装上の注意：14体のエージェントでは全91組を計算し、各組の大きさの時系列を監視する。多くの組が同時に増えることを、創発の予兆として扱う。

第4章　臨界減速の兆候（第8章）

回復の速さ λ をゼロに近づけていく、最も簡単な自己回帰のモデルで、ゆらぎの分散と1次の自己相関が増えることを確かめる。

```
import numpy as np
rng=np.random.default_rng(2)
# 臨界減速のおもちゃモデル：x_{t+1} = (1-λ) x_t + ノイズ。回復の速さ λ が 0 に近づく（固有値がゼロへ）
T=4000; lam=np.linspace(0.5,0.02,T); x=np.zeros(T)
for t in range(T-1): x[t+1]=(1-lam[t])*x[t]+rng.normal(0,0.1)
for a,b in [(0,1000),(1000,2000),(2000,3000),(3000,4000)]:
    s=x[a:b]; ac=np.corrcoef(s[:-1],s[1:])[0,1]
    print(f"区間 {a:4d}-{b:4d}: 平均λ={lam[a:b].mean():.3f}  分散={s.var():.4f}  1次の自己相関={ac:.3f}")
```

出力：λ が0.44から0.08へ下がるにつれ、分散は0.0145から0.0730へ、自己相関は0.541から0.928へ増える。固有値を直接観測できないとき、この2つが臨界減速の実用的な兆候になる。

第5章　相互情報量、データ処理不等式、KL情報量（第12章）

```
import numpy as np
rng=np.random.default_rng(0)
# 現実 X（安定=0 / 不安定=1）と観測 Y（6段階の指標）。不安定ほど大きな値が出やすい
pX=np.array([0.8,0.2])
pY_X=np.array([[.30,.30,.20,.12,.06,.02],   # X=0
               [.02,.05,.13,.20,.25,.35]])  # X=1
def MI(pX,pY_X):
    pXY=pX[:,None]*pY_X; pY=pXY.sum(0)
    m=pXY>0; return (pXY[m]*np.log2(pXY[m]/(pX[:,None]*pY[None,:])[m])).sum()
# 検閲：大きな値（不都合なデータ，Y>=4）を消して「4未満の最大値3」に書き換える（Yだけを見た後処理）
C=np.zeros((6,6));
for y in range(6): C[y,min(y,3)]=1
pYc_X=pY_X@C
print(f"I(X;Y)          = {MI(pX,pY_X):.3f} bit")
print(f"I(X;Y_censored) = {MI(pX,pYc_X):.3f} bit   （データ処理不等式により必ず小さいか等しい）")
# KL情報量：真の分布と、検閲データで学習したモデル分布
P_true=pX@pY_X; Q_model=pX@pYc_X+1e-12
KL=lambda P,Q:(P[P>0]*np.log2(P[P>0]/Q[P>0])).sum()
print(f"D_KL(P_true||Q_model) = {KL(P_true,Q_model):.3f} bit")
print(f"D_KL(Q_model||P_true) = {KL(Q_model,P_true+1e-12):.3f} bit   （順番を入れ替えると値が変わる）")
```

出力：$I(X;Y) = 0.247$ビット、$I(X;Y_{\text{censored}}) = 0.203$ビット（データ処理不等式のとおり減少）。$D_{\mathrm{KL}}(P_{\text{true}} \| Q_{\text{model}}) = 6.534$ビット、$D_{\mathrm{KL}}(Q_{\text{model}} \| P_{\text{true}}) = 0.395$ビットで、順番によっ

て値が大きく変わる。前者が大きいのは、モデルが真の分布の確率をほとんどゼロとみなす値（不都合なデータ）を含むためである。
実装上の注意：実データでは確率がゼロの区間が生じるので、平滑化を行う。KL情報量は距離ではないことに注意する。

第6章　偏りのある確率的勾配降下法（第13章）

```
import numpy as np
rng=np.random.default_rng(1)
# 最小化する関数 f(w) = |w - w*|^2 / 2,  真の最適点 w* = (1, 1)
w_star=np.array([1.0,1.0])
def run(bias, steps=20000):
    w=np.array([-2.0,3.0])
    for t in range(1,steps+1):
        g=(w-w_star)+rng.normal(0,1.0,2)+bias   # ノイズのある勾配（＋固定の偏り）
        w-=g/(t+10)**0.7                         # 学習率を少しずつ下げる
    return w
print("偏りのないノイズ      :", np.round(run(np.zeros(2)),3), "（真の最適点 (1,1) の近く）")
print("向きの固定された偏り  :", np.round(run(np.array([0.5,0.0])),3), "（(0.5,1) 付近：ずれた点に落ち着く）")
```

出力：偏りのないノイズでは $(0.989, 0.986)$（真の最適点 $(1, 1)$ の近く）、向きの固定された偏り $(0.5, 0)$ がある場合は $(0.499, 1.024)$ に落ち着く。学習率を下げても、偏りによるずれは消えない。

付録 A　参照実装

本付録のコードは、本文の「実装上の注意」で述べた点を示すための最小限の例であり、本編の数値を再現するものではない。いずれも実際に実行し、出力を確認した。

A.1　集計・小さなセルの秘匿・差分プライバシー

```
import numpy as np

def aggregate_monthly(records, months, categories, k_min=5, epsilon=1.0, seed=0):
    """個別の記録（月, 区分）を月次・区分ごとの件数にまとめ、小さなセルを秘匿し、
    ラプラス・ノイズで差分プライバシーを付ける（1 件の増減で件数は最大 1 変わる）。"""
    rng = np.random.default_rng(seed)
    counts = np.zeros((len(months), len(categories)))
    for m, c in records:
        counts[months.index(m), categories.index(c)] += 1
    suppressed = counts < k_min                       # 小さなセルの秘匿
    noisy = counts + rng.laplace(0.0, 1.0 / epsilon, counts.shape)
    noisy[suppressed] = np.nan                        # 伏せた区分は欠測として扱う
    return np.round(noisy, 1), suppressed

months = ['2026-04', '2026-05', '2026-06']
cats = ['接触', '訪問', '通信']
table = {('2026-04', '接触'): 12, ('2026-04', '訪問'): 9, ('2026-04', '通信'): 3,
         ('2026-05', '接触'): 15, ('2026-05', '訪問'): 11, ('2026-05', '通信'): 8,
         ('2026-06', '接触'): 21, ('2026-06', '訪問'): 2,  ('2026-06', '通信'): 10}
records = [key for key, n in table.items() for _ in range(n)]   # 個別の記録（例）
agg, sup = aggregate_monthly(records, months, cats)
print(agg)
print('秘匿したセルの数:', int(sup.sum()))
```

実行結果（ノイズは乱数の種を固定して生成）：

```
[[12.3  8.4  nan]
 [11.6 12.   9.7]
 [21.2  nan 10.1]]
秘匿したセルの数: 2
```

　件数が 5 未満の区分（2026 年 4 月の「通信」3 件、2026 年 6 月の「訪問」2 件）は伏せられ、それ以外の件数にはノイズが加わっている。多様体には、このような集計指標だけが入る。

A.2　層ラプラシアンと頂点ごとのスコア

```
import numpy as np

def sheaf_laplacian(n_nodes, stalk_dim, edges):
```

```
    """edges: [(u, v, F_u, F_v)]。F_u, F_v は頂点のデータを辺の上で比べる制限写像（行列）。"""
    rows = []
    for u, v, Fu, Fv in edges:
        r = np.zeros((Fu.shape[0], n_nodes * stalk_dim))
        r[:, u*stalk_dim:(u+1)*stalk_dim] = -Fu
        r[:, v*stalk_dim:(v+1)*stalk_dim] = Fv
        rows.append(r)
    delta = np.vstack(rows)          # 余境界作用素 δ
    return delta, delta.T @ delta    # 層ラプラシアン L = δ^T δ

def node_scores(L, x, n_nodes, stalk_dim):
    """全体のデータ x に L を一度だけ作用させ、頂点ごとのブロックの大きさをスコアにする。"""
    Lx = (L @ x).reshape(n_nodes, stalk_dim)
    return Lx, np.linalg.norm(Lx, axis=1)

# 3 部署の演習問題：A(営業), B(物流), C(経理)。各部署は（数量, 単価）を持つ
I = np.eye(2)
edges = [(0, 1, I, I), (1, 2, I, I), (2, 0, I, I)]   # 両端の記録は一致すべき
x = np.array([10, 50,  10, 50,  10, 80], float)
delta, L = sheaf_laplacian(3, 2, edges)
Lx, score = node_scores(L, x, 3, 2)
print("L x =", Lx.tolist())
print("スコア =", np.round(score, 1).tolist(), " 整合性半径 =", round(float(np.linalg.norm(delta @ x)), 2))
```

実行結果：

```
L x = [[0.0, -30.0], [0.0, -30.0], [0.0, 60.0]]
スコア = [30.0, 30.0, 60.0]  整合性半径 = 42.43
```

組織全体のデータを 1 つのベクトルに並べ、層ラプラシアンを一度だけ作用させて、頂点ごとのブロックの大きさをスコアにしている。補助文書 02 第 12.7 節の演習問題と同じ結果である。

付録 B　著者による関連記事

本作の数学とAIの場面は、著者（Étale Cohomology）がQiita、note、Zennで公開してきた次の記事と書籍を下敷きにしている。いずれも、本書の補助文書の執筆時点（2026年10月）で公開されているものである。

Qiita

[Q1] 「14体のAIが勝手に”同期・雪崩・フラクタル”を起こした話（創発の数理を全部解説する）」Qiita、2026年3月25日公開。
https://qiita.com/etale_cohomology/items/cf5e1657f3440eabf4e8
同じ多様体の上で14体のAIエージェントを同時に動かしたとき、位相同期（蔵本モデル）、カスケード（自己組織化臨界）、フラクタルという3つの創発現象が現れ、その源がベクトル場のLie括弧の非可換性にあることを解説する。段階的な対処（警報、他のエージェントの影

響の推定、モース理論による地形の変化の予測、平均場ゲーム）と、顧客への正直な報告の原則も示す。本作第5章・第9章の土台。

[Q2] 「『実データで動かしたら、曲率マップが教科書と全然違った』—— 手術理論の適用限界と、5つの幾何学的拡張ロードマップ」Qiita、2026年3月26日公開。
https://qiita.com/etale_cohomology/items/6461c6e73a1e43981636
古典的な手術理論は任意の形の多様体の位相の変化を予測する理論ではなく、実際に使えるのはモース理論と分岐（臨界点のbirth-death）であることを整理する。実データでの品質検査の不合格と、最適輸送、情報幾何、ポアンカレ埋め込み、シンプレクティック幾何、ワイル幾何の5つの拡張の道筋を示す。本作第8章の土台。

[Q3] 「『企業のロビイングって、手術理論を意図的に使っているのでは？』—— シンプレクティック幾何と保存量の破れ」Qiita、2026年3月27日公開。
https://qiita.com/etale_cohomology/items/6ee3e04917e7a712753d
主体が意図して地形の位相を変える行為を3段階（機会の発見、臨界点を動かすベクトル場の設計、副作用の事前シミュレーション）で定式化し、シンプレクティック積分器による長期シミュレーションの安定性と、保存量の破れによる異常の検知を解説する。倫理的なガードレールの必要性にも触れる。本作第10章の土台。

[Q4] 「【ジオメトリック・インテリジェンス 番外編】（第5弾）『Etale』の防壁 —— 形式検証はどこに必要か」Qiita、2026年3月27日公開。
https://qiita.com/etale_cohomology/items/8f824717142ede8b5a86
形式検証を、失敗したときに取り返しのつかない5つの部分（VAEのフルランク性、シンプレクティック積分器の保存量、手術の設計の安全性、エージェント間の利益相反、層による改ざんの検知）に絞る考え方と、Coq（Flocq、ONNX経由の検証済み変換）、Lean 4、Agda、Z3、Isabelle/HOLの使い分けを示す。小説の中の架空のツールと実在のツールを区別する注意も述べる。本作第6章の土台。

[Q5] 「そのM&A、数学的に間違っていませんか？多様体の曲率が暴く統合リスク【理論編】」Qiita、2026年3月17日公開。
https://qiita.com/etale_cohomology/items/dc9b133f7658a9a23ef6
2つの多様体の接合（連結和とGromov–Lawsonの構成、ファイバー束、VAEの共同学習）を5段階の読者層に向けて解説し、手術理論をM&Aの効果の予測にそのまま使えない3つの理由を示す。本作第8章（同盟の構築・離脱をM&Aに重ねる議論）の参考。

[Q6] 「そのM&A、数学的に間違っていませんか？多様体の曲率が暴く統合リスク【実践ケース・スタディ編】」Qiita、2026年3月17日公開（同年10月25日更新）。
https://qiita.com/etale_cohomology/items/98bb7be6c9bfeab83606
架空の企業の買収を舞台に、幾何学的デューデリジェンス、2つの多様体の比較、統合多様体の構築と接合部の断層の検出を、物語の形で示す。

note

[N1] 「自動運転車や介護ロボットの事故を、どう防ぐか —— OpenAIやGoogleが注目する『形式証明』とAI安全性の最前線」note、2026年6月22日公開。
https://note.com/etale_cohomology/n/nbcd747eddfe4
テストと形式手法の補完関係、主要国と主要企業のAIの安全性の取り組み、経営者が取るべき行動を、経営者向けに整理する。

[N2] 「『経済安全保障』とは何か —— 業界別×企業規模別に読む『取締役会の法的義務』と、投資審査・制裁というグローバル・ルール」note、2026年7月11日公開。
https://note.com/etale_cohomology/n/n0b8ee96f4855
経済安全保障推進法の4つの制度、外為法、セキュリティ・クリアランス制度、米国の対米外国投資委員会と二次制裁、中国の対抗立法などを、業界と企業規模ごとに整理する。

Zenn

[Z1] 『ジオメトリック・インテリジェンス』（日本語版）https://zenn.dev/etalecohomology/books/3761995ca27127／（英語版）https://zenn.dev/etalecohomology/books/9f680ca9dddf2a
経営環境多様体の構築からLie微分、最適制御までの理論とPythonの実装をまとめた書籍。本作の数学の出発点。

[Z2] 『「個別は正常、全体は矛盾」を数学で解く ～層コホモロジーによるITインフラ監視入門～』https://zenn.dev/etalecohomology/books/e5457a9d656717

[Z3] 『データの「つなぎ目」の矛盾・不整合を「層理論AI」で検出する』https://zenn.dev/etalecohomology/books/e24aaec20b53cd

[Z4] 『ÉTALE Shield 層ラプラシアンの防壁』https://zenn.dev/etalecohomology/books/245d38d9601b5a
[Z2]～[Z4]は、層理論と層ラプラシアンによる整合性の検証を、ITインフラの監視やデータの不整合の検出に応用する書籍。本作のNeural Sheaf Diffusionと整合性の半径の議論の背景。

[Z5] 『論理学から AI Safety へ —— 圏論・Gödel・HoTT がつなぐ知の地図』https://zenn.dev/etalecohomology/books/11785c01a97658
形式手法の数学的な基礎（論理、型、圏論）を解説する書籍。本作のZKMLと形式検証の議論の背景。